TicketChain — Week 2 Progress Report

CS414 Fundamentals of Blockchain · Localized P2P Event Ticketing Network Team: David · Haythem · Khalil · Mahmoud

1. Summary

Week 2's QA task was to make the anti-scalping logic provably correct and to characterize the system's real performance rather than its assumed performance. We expanded the `TicketNFT` contract test suite from 14 to 25 tests (100% statement/branch coverage), wrote a step-by-step live-demo runbook for both the Sunny Day and Rainy Day scenarios, and built a reproducible benchmark harness for transaction execution time, PoW block finality, and P2P discovery/propagation latency. Running that harness for real — rather than estimating the numbers — surfaced four genuine bugs in the P2P broadcast path that would otherwise have surfaced live during a demo; all four are fixed and verified in this report.

2. Smart Contract Testing

`contracts/test/TicketNFT.test.js` grew from 14 to **25 tests**, adding coverage the original suite didn't reach:

- **Minting** — sequential token IDs across multiple mints; minting to the zero address reverts.
- **`setResellable` access control** — only the organizer may lock/unlock a ticket; reverts for a nonexistent ticket.
- **`resaleTransfer` edge cases** — reverts for a nonexistent ticket; supports being resold multiple times in a row, always at face value; reverts and leaves ownership unchanged when the payment forwarding call to the seller fails (a new `RejectsEther` mock contract exercises this path, since no EOA can trigger it).
- **Unauthorized transfers** — a stranger with no approval, a per-token approved address, and an operator approved via `setApprovalForAll` are all blocked from moving a ticket via raw `transferFrom`; the anti-scalping lockdown in `_update` fires before ERC-721's own authorization check even runs, so approval can never be used to route around the price ceiling.

```
25 passing (722ms)
```

File	% Stmts	% Branch	% Funcs	% Lines
`TicketNFT.sol`	100	100	100	100
`mocks/RejectsEther.sol`	100	100	100	100

100% statement, branch, function, and line coverage on `TicketNFT.sol`, measured with `npx hardhat coverage`.

3. Live Demo Scenarios

Full step-by-step procedures — MetaMask setup, exact commands, and actual captured output — are documented in `docs/demo-scenarios.md`. Summary:

- **Sunny Day** — a buyer purchases a ticket through the React UI at exactly the on-chain face value. Verified end-to-end: 24 ms submit→receipt, 78,534 gas, ownership and price both correct afterward.
- **Rainy Day** — a scalper attempts to buy at 3× face value. Since the UI can only ever submit the exact on-chain price, this has to be demonstrated with a direct contract call — `contracts/scripts/demo-rainy-day.js` does that and prints the revert live. Verified: the transaction reverts with `"Scalping detected: Price exceeds face value"` and ticket ownership is unchanged.

The demo doc also flags a real integration gap worth stating to the class rather than hiding: a UI purchase is not currently broadcast into the P2P layer, so the "Available Events" feed keeps showing a ticket's original listing after it sells. On-chain state (ownership, price) is always correct; only the P2P availability feed lags behind actual on-chain sales.

4. Metrics

Methodology and the runnable harness: `network/benchmark.py` (`python benchmark.py --p2p`), plus the isolated on-chain timing script used for the Sunny Day numbers. All figures below are measured on one developer machine (not a fixed reference machine), across 3–10 trials per metric — treat these as characterizing order of magnitude, not tight SLAs.

Metric	Result	Method
On-chain purchase (`resaleTransfer`), submit → receipt	24 ms, 78,534 gas	Local Hardhat node, direct script call
Off-chain tx sign + validate (blockchain core)	~0.77–0.86 ms avg (200 trials/run)	`benchmark.py bench_transaction_execution`
PoW block finality, difficulty = 16 bits, 5 tx/block	~0.24–0.58 s avg per 10-trial batch (individual mines: 16 ms–2.0 s)	`benchmark.py bench_block_finality`
P2P peer discovery latency (2 local IPv8 nodes)	~0.33–0.56 s (3 runs)	`benchmark.py --p2p`, UDP broadcast bootstrap
P2P block propagation, mine → peer validates & appends	~0.15–1.4 s (3 runs; dominated by the PoW step above)	`benchmark.py --p2p`, end-to-end

Block-finality variance is expected and by design: PoW is a random search, so mining time follows a geometric-ish distribution around the difficulty target rather than a fixed cost — the min/max spread (16 ms to 2.0 s across runs) is the mechanism working as intended, not noise to average away.

5. Bugs Found and Fixed During QA

Actually running the P2P benchmark — instead of trusting that the broadcast path worked because the code read correctly — surfaced four real defects, all now fixed and covered by a passing end-to-end run (`network/benchmark.py --p2p`: peer discovery, tx broadcast, block mining, and remote block acceptance all succeed):

1. **Wrong wire-format packer name** (`network/main.py`). `TransactionPayload` and `BlockPayload` declared their length-prefixed byte field as `"4?H"`, which pyipv8's serializer doesn't register — every broadcast raised `PackError` before a single byte left the node. Fixed to `"varlenH"` / `"varlenI"`.
2. **Missing `@lazy_wrapper` decorator** (`network/main.py`). `_on_transaction` and `_on_block` were registered as raw message handlers but written as if pyipv8 would hand them a decoded `Peer`/payload pair automatically — it doesn't; `add_message_handler` delivers raw `(address, bytes)` unless the handler is wrapped with `@lazy_wrapper(PayloadClass)`. Every inbound message crashed with `AttributeError: 'bytes' object has no attribute 'puzzle_ok'`.
3. **Non-deterministic genesis block** (`network/blockchain/block.py`). `genesis_block()` timestamped itself with `time.time()`, so every independently-started node mined a genesis block with a different hash. Since block #1's `prev_hash` is the sender's genesis hash, no block from one node could ever pass a peer's chain-linkage check — P2P block propagation was structurally impossible, not just occasionally flaky. Fixed by pinning the genesis timestamp to a shared constant, matching how real chains (e.g. Bitcoin) hardcode their genesis rather than deriving it from wall-clock time.
4. **Non-ASCII console output crash on Windows** (`network/main.py`). Status logs use →/…, which the default Windows console codepage (cp1252) can't encode; a `print()` mid-handler raised `UnicodeEncodeError`, which silently ate the rest of that handler's work. Fixed by forcing UTF-8 stdout/stderr at startup.

None of these were visible from reading the code or from the existing `test_two_nodes.py` (which only exercises peer discovery, not message payloads) — they only showed up once transactions and blocks were actually sent over the wire between two processes.

6. Individual Contributions (Week 2)

- **Khalil** — QA task: expanded the contract test suite to 25 tests with 100% coverage, wrote the demo-scenarios runbook and the Rainy Day demo script, built and ran the performance benchmark harness, found and fixed the four P2P bugs in Section 5, and compiled this report.
- **David** — Ticket resale logic refinements and local development environment support (`start_dev.sh`, deploy-time ABI export).
- **Mahmoud** — HTTP API for the frontend, localized peer-community broadcasting (event-scoped overlays), and the per-message search-puzzle Sybil-resistance mechanism.
- **Haythem** — Finalized the on-chain anti-scalping transfer logic and ABI export pipeline; live contract-call wiring for the purchase flow.

7. Current Challenges

- The P2P layer and the on-chain contract remain two separate sources of truth: a real purchase doesn't update the P2P "availability" feed (see Section 3). Bridging `TicketTransferred` events into `TicketChainCommunity.submit_transaction` is the natural next step. Ordering matters here — the reason a resale is at all discoverable in the present grid is because `main.py`'s `seed_tickets()` seeds the P2P chain once at startup, not from live contract events.
- `pyipv8` 2.13's `vp_compile()` relies on writing into `locals()` inside an `exec()` call, a pattern PEP 667 (Python 3.13+) breaks. We work around it in `benchmark.py` for the benchmark process; `main.py` itself (used by `start_dev.sh`) has not needed the patch yet because the team's dev machines run an older Python — worth pinning a Python version constraint before anyone upgrades and hits this blind.
- No automated test exercises the actual wire serialization of `TransactionPayload`/`BlockPayload` (bug #1 in Section 5 shipped undetected for exactly this reason) — worth adding a packet round-trip test alongside `test_two_nodes.py`.

8. Plan for Week 3

- Bridge on-chain `TicketTransferred` events into the P2P layer so the availability feed reflects real sales.
- Add a wire-level pack/unpack regression test for `TransactionPayload` and `BlockPayload` to prevent a repeat of bug #1.
- Extend the benchmark harness to a multi-node (3+) topology to see how discovery/propagation latency scales past a two-node pair.
- Deploy the P2P node + contract stack somewhere reachable for a real cross-machine demo, rather than localhost-only.